University of Wah

Department of Computer Science

BS Cyber Security

Internet of Things

Project Report

Title: IoT-Based Smart RFID User Management and Access Control System Using ESP32

Submitted to: Sir Hammad Ali

Group Members:

Hassan Iftikhar (UW-23-CY-BS-002)

Muhammad Azfar Waqas (UW-23-CY-BS-013)

Usman Kaleem (UW-23-CY-BS-015)

Albash Ahmad (UW-23-CY-BS-024)

Muhammad Hassan (UW-23-CY-BS-035)

Section: BS-CYS-6N

Table of Contents

Abstract

The rapid advancement of IoT technologies has increased the demand for smart, secure, and scalable access control systems. This project presents an IoT-Based Smart RFID User Management and Access Control System developed using the ESP32 microcontroller. The system authenticates users via RFID cards, displays real-time status on an OLED screen, logs authentication events to an SD card, and provides a live web dashboard for user management and monitoring. By integrating RFID authentication, SPI communication, I2C display, WiFi networking, and filesystem management, the system demonstrates a comprehensive embedded security solution suitable for academic and small-scale enterprise deployment.

Keywords: *ESP32, RFID Authentication, IoT Security, Access Control, OLED Display, SD Card Logging, Web Dashboard, SPI, I2C, LittleFS*

1 Introduction:

Modern access control systems have evolved significantly from traditional lock-and-key mechanisms to sophisticated electronic and IoT-enabled solutions. With the proliferation of connected devices, there is a growing need for intelligent systems that can authenticate users, log events, and provide real-time monitoring through web interfaces.

This project, titled IoT-Based Smart RFID User Management and Access Control System Using ESP32, addresses this need by combining embedded hardware with wireless networking capabilities. The system leverages the ESP32 microcontroller's dual-core processing and built-in WiFi to create a fully functional, self-hosted access control solution.

1.1 Purpose of this Project:

The primary purpose of this project is to develop a secure, smart, and real-time access control system using IoT technologies. The system aims to authenticate RFID cards, provide immediate visual and audio feedback, maintain persistent logs of all authentication events, and offer a web-based dashboard for centralized user management and monitoring.

1.2 Motivation & Problem Statement:

Traditional access control systems often lack real-time monitoring, remote management capabilities, and persistent logging. Physical key-based systems provide no audit trail and are difficult to manage at scale. The motivation behind this project is to bridge this gap by creating an affordable, ESP32-based solution that brings enterprise-grade access control features to academic and small-scale environments.

Key problems addressed include: lack of real-time user authentication feedback, absence of centralized access logs, inability to manage users remotely, and the need for a cost-effective embedded security solution.

1.3 Scope of Project:

The scope of this project is confined to RFID-based user authentication using the RC522 module, real-time status display via an SSD1306 OLED, SD card-based event logging, and a WiFi-hosted web dashboard for user management. The system operates within a local network environment and is intended for educational demonstration and small-scale deployment.

1.4 Platform Overview: ESP32:

The ESP32-WROOM-32D serves as the core processing unit. It was selected for its integrated dual-core processor, built-in WiFi and Bluetooth capabilities, multiple SPI/I2C interfaces, large flash memory, and suitability for IoT applications. Its low cost and extensive library support within the Arduino ecosystem make it ideal for rapid prototyping and academic projects.

2 Overall Description:

2.1 System Overview:

The system is a self-contained IoT access control unit that combines RFID card reading, real-time display feedback, persistent SD card logging, and a WiFi-hosted web dashboard. Upon startup, the ESP32 connects to a configured WiFi network, initializes all peripherals, and enters an active monitoring loop awaiting RFID card scans. All components operate concurrently, with authentication events triggering display updates, LED/buzzer alerts, log writes, and dashboard refreshes simultaneously.

2.2 User Classes & Characteristics:

The system serves the following user classes:

- System Administrator: Manages user database, adds/removes RFID cards via web dashboard
- Authorized Users: Individuals holding registered RFID cards who gain physical access
- Security Auditors: Personnel reviewing authentication logs stored on the SD card
- Academic Researchers: Students and instructors studying IoT security and embedded systems

2.3 Assumptions & Design Constraints:

- The system operates within a local WiFi network environment
- Hardware resources are constrained to the ESP32 platform
- RFID cards operate at 13.56 MHz using the ISO/IEC 14443 standard
- SD card is formatted as FAT32 for filesystem compatibility
- Web dashboard is accessible only within the local network

2.4 Operational Environment:

The system is designed for indoor environments such as classrooms, laboratories, offices, and residential spaces. It operates on 5V USB power and communicates over a standard 2.4 GHz WiFi network. The web dashboard is accessible from any device on the same network using the ESP32's assigned IP address.

3 Technical Background:

3.1 RFID Technology Overview:

Radio Frequency Identification (RFID) is a wireless technology that uses electromagnetic fields to automatically identify and track tags attached to objects. The RC522 module used in this project operates at 13.56 MHz and communicates via the SPI protocol. Each RFID card contains a unique UID (User Identifier) that the system reads and validates against a stored user database.

3.2 SPI Communication Protocol:

Serial Peripheral Interface (SPI) is a synchronous serial communication protocol used for short-distance communication. It uses four lines: MOSI (Master Out Slave In), MISO (Master In Slave Out), SCK (Serial Clock), and SS/CS (Slave Select/Chip Select). The ESP32 supports two independent SPI buses (VSPI and HSPI), which are utilized in this project to avoid bus conflicts between the RFID module and SD card module.

3.3 I2C Communication Protocol:

Inter-Integrated Circuit (I2C) is a two-wire serial protocol using SDA (data) and SCL (clock) lines. The SSD1306 OLED display communicates via I2C, connected to GPIO 21 (SDA) and GPIO 22 (SCL) on the ESP32. I2C allows multiple devices to share the same bus using unique device addresses.

3.4 ESP32 WiFi & Web Server:

The ESP32 integrates a dual-mode WiFi radio supporting IEEE 802.11 b/g/n standards. The ESPAsyncWebServer library enables hosting of asynchronous HTTP web servers directly on the ESP32. Combined with LittleFS for serving static web files, this allows a full web dashboard to be hosted without external server hardware.

3.5 Security Considerations in RFID Systems:

RFID-based systems face several security challenges including card cloning, replay attacks, and unauthorized scanning. Mitigations include using cards with encrypted memory sectors, maintaining a centralized revocation list, and implementing session tokens. This project focuses on UID-based authentication as a foundational layer, with acknowledgment that production deployments should implement additional cryptographic protections.

4 System Architecture:

4.1 Hardware Architecture:

The hardware architecture centers on the ESP32-WROOM-32D microcontroller, which interfaces with multiple peripheral modules via SPI and I2C buses. The architecture deliberately separates the RC522 RFID module and SD card module onto independent SPI buses to eliminate bus contention.

Component	Interface	Purpose
ESP32-WROOM-32D	Central	Main microcontroller & WiFi
RC522 RFID Module	VSPI (SPI)	Card reading & authentication
SSD1306 OLED (128x64)	I2C	Status & user info display
SD Card Module	HSPI (SPI)	Authentication log storage
Active Buzzer	GPIO 4	Audio alert feedback
LED + 220Ω Resistor	GPIO 26	Visual access indication
RFID Cards/Tags	RF 13.56MHz	User authentication tokens

4.2 Pin Configuration:

RFID RC522 (VSPI)

Signal	GPIO Pin	Description
SDA/SS	GPIO 5	Slave Select
SCK	GPIO 18	SPI Clock
MOSI	GPIO 23	Master Out Slave In
MISO	GPIO 19	Master In Slave Out
RST	GPIO 27	Reset
VCC	3.3V	Power Supply
GND	GND	Ground

SD Card Module (HSPI)

Signal	GPIO Pin	Description
CS	GPIO 16	Chip Select
SCK	GPIO 14	SPI Clock
MOSI	GPIO 13	Master Out Slave In
MISO	GPIO 25	Master In Slave Out
VCC	5V	Power Supply
GND	GND	Ground

OLED Display (I2C)

Signal	GPIO Pin	Description
SDA	GPIO 21	I2C Data
SCL	GPIO 22	I2C Clock
VCC	3.3V	Power Supply
GND	GND	Ground

4.3 SPI Bus Architecture:

A critical architectural decision was separating the two SPI devices onto independent buses to resolve communication conflicts:

SPI Bus	Used For	SCK	MOSI	MISO
VSPI	RFID RC522	GPIO 18	GPIO 23	GPIO 19
HSPI	SD Card Module	GPIO 14	GPIO 13	GPIO 25

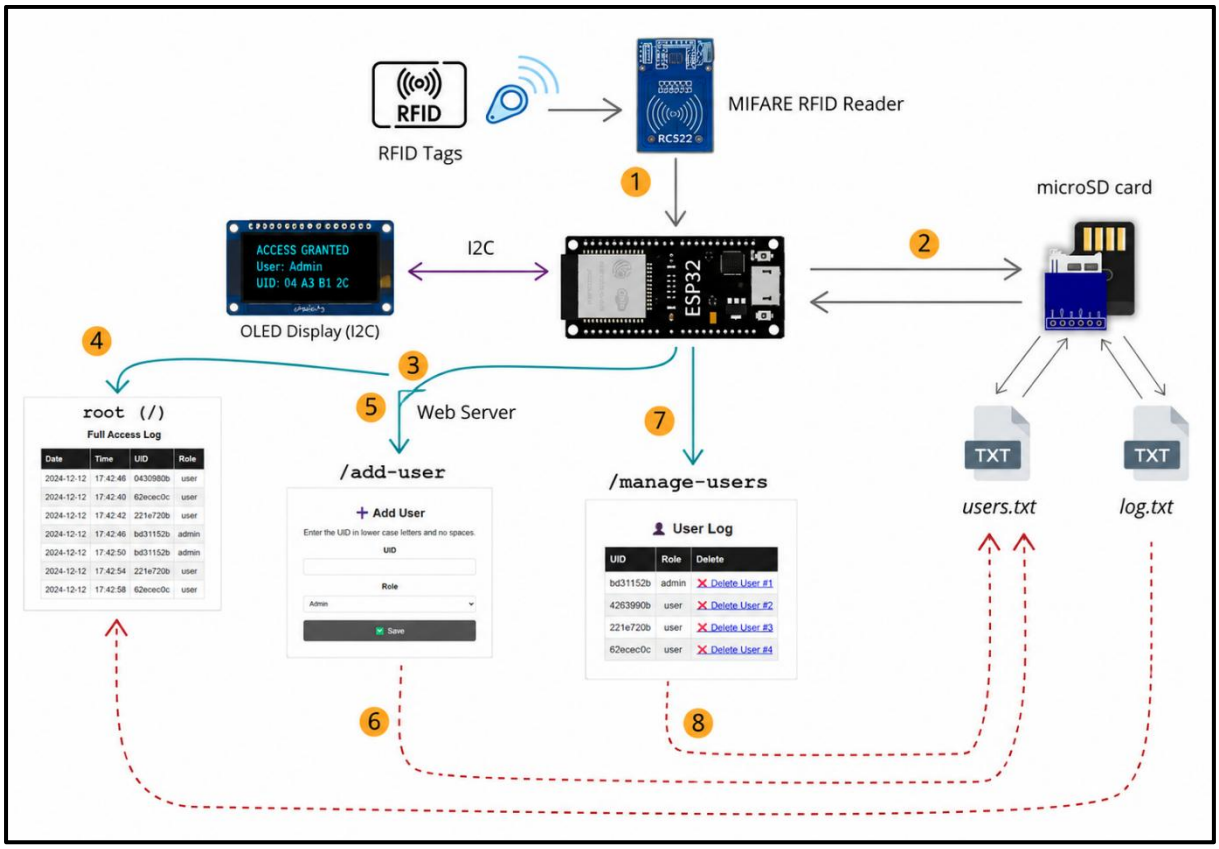
4.4 Software Architecture:

The software is developed using the Arduino framework for ESP32 and follows a modular layered architecture. Key software layers include: Hardware Abstraction Layer (SPI/I2C drivers), Authentication Module (UID verification), Display Module (OLED rendering), Logging Module (SD card writes), Web Server Module (dashboard hosting), and Control Logic (event coordination).

4.5 Communication & Data Flow:

The data flow begins when a user presents an RFID card. The RC522 module reads the UID via VSPI and passes it to the ESP32. The authentication module checks the UID against the user database stored in LittleFS. Based on the result, the OLED display updates, LED and buzzer activate, the event is logged to the SD card via HSPI, and the web dashboard refreshes in real time via WiFi.

Diagram:



5 Functional Requirements

5.1 RFID Authentication:

The system shall continuously scan for RFID cards/tags using the RC522 module. Upon card detection, it shall read the UID and verify it against the registered user database. The system shall respond within milliseconds with an access granted or denied determination.

5.2 OLED Display Interface:

The system shall display real-time status information on the SSD1306 OLED screen. Required display screens include: System Ready, Card Detected, Access Granted (with user name and role), Access Denied, and Date/Time display. The display shall update immediately upon each authentication event.

5.3 SD Card Event Logging:

The system shall log all authentication events to the SD card with timestamps, UID, user name, role, and access result. Logs shall persist across power cycles and shall be retrievable for security audit purposes. Log entries shall be stored in a structured format for easy parsing.

5.4 Web Dashboard:

The system shall host a web dashboard accessible via WiFi on the local network. The dashboard shall provide: user management (add/remove registered cards), real-time authentication log viewing, RFID monitoring, and system status display. The dashboard shall update in real time without requiring manual page refresh.

5.5 LED and Buzzer Alerts:

The system shall provide immediate audiovisual feedback upon authentication events. The LED (GPIO 26) and buzzer (GPIO 4) shall activate with distinct patterns for access granted and access denied scenarios, providing unambiguous physical-layer feedback independent of the display or network.

5.6 WIFI Connectivity & Web Server:

The ESP32 shall connect to a configured WiFi network on startup and host an asynchronous web server using ESPAsyncWebServer. Static web files (HTML, CSS, JavaScript) shall be served from LittleFS flash storage. The system shall maintain stable WiFi connectivity during normal operation.

6 Non-Functional Requirements

6.1 Performance Requirements:

Authentication processing shall complete within 500 milliseconds of card presentation. OLED display updates shall occur within 100 milliseconds of an authentication event. Web dashboard shall reflect authentication events within 2 seconds. The system shall operate continuously without memory leaks or crashes.

6.2 Accuracy & Reliability:

The RFID authentication module shall achieve 100% accuracy in UID reading under normal operating conditions. The system shall correctly classify all registered UIDs as authorized and all unregistered UIDs as unauthorized. SD card logging shall achieve 100% write reliability under normal operation.

6.3 Usability:

The OLED display shall present information in a clear, readable format. The web dashboard shall be intuitive for users with basic web browsing skills. Card registration and removal via the dashboard shall require no programming knowledge.

6.4 Scalability:

The user database shall support storage of multiple registered RFID UIDs limited by available LittleFS flash storage. The SD card logging system shall support long-term log storage limited only by SD card capacity. The modular software architecture shall allow addition of new features with minimal changes.

6.5 Security Considerations:

The web dashboard shall be accessible only within the local network. User database modifications shall require authenticated access. The system shall not expose sensitive user data via unprotected API endpoints. All authentication events shall be logged for audit trail purposes.

7 Implementation Details

7.1 System Initialization:

On power-up, the firmware executes the following initialization sequence: (1) Serial communication begins for debugging, (2) LittleFS is mounted for web file serving, (3) SSD1306 OLED is initialized via I2C showing a system ready message, (4) RC522 RFID module is initialized via VSPI, (5) SD card module is initialized via HSPI, (6) WiFi connection is established, (7) ESPAsyncWebServer is started, and (8) the main authentication loop begins.

7.2 RFID Authentication Logic:

The authentication loop continuously polls the RC522 module for card presence. Upon card detection, the UID is extracted as a hexadecimal string and compared against entries in the user database (stored in SD card storage as a text-based user database file). A match results in Access Granted with the associated user name and role being retrieved; no match results in Access Denied. The result triggers corresponding OLED updates, LED/buzzer activation, and log writes.

7.3 Dual SPI Bus Management:

The critical implementation challenge of running both RC522 and SD card simultaneously was resolved by assigning each to a separate ESP32 SPI bus. The MFRC522 library was configured to use VSPI (default ESP32 SPI) while the SD library was initialized with a custom SPIClass instance on HSPI pins. Each module's Chip Select pin is managed independently, preventing bus contention during concurrent operations.

7.4 OLED Display Management:

The Adafruit SSD1306 and GFX libraries render text and graphics on the 128x64 OLED display. Display screens are implemented as functions: showReady(), showCardDetected(uid), showAccessGranted(name, role), showAccessDenied(), and showDateTime(). Each function clears the display buffer, renders the appropriate content, and calls display.display() to push the buffer to the screen.

7.5 SD Card Logging:

Authentication events are appended to a CSV log file on the SD card. Each log entry includes: timestamp (obtained from an NTP client or RTC), UID, user name (or UNKNOWN for unregistered cards), access result (GRANTED/DENIED), and a sequential event number. The log file is opened in append mode for each write operation and closed immediately after to prevent data corruption.

7.6 Web Dashboard Implementation:

The web dashboard is built as a single-page application with HTML, CSS, and JavaScript files stored in LittleFS. The ESPAsyncWebServer handles HTTP GET requests for the dashboard and exposes REST-like API endpoints for: fetching current user list (/api/users), adding a new user (/api/add), removing a user (/api/remove), and retrieving recent logs (/api/logs). The dashboard uses JavaScript fetch() calls to poll these endpoints and update the UI in real time.

8 Tools & Technologies

8.1 Hardware Components:

Component	Specification	Purpose
ESP32-WROOM-32D	Dual-core 240MHz, 4MB Flash	Main microcontroller & WiFi
RC522 RFID Module	13.56 MHz, ISO 14443	RFID card reading
SSD1306 OLED	128x64 pixels, I2C	Status display
SD Card Module	SPI interface, FAT32	Event log storage
Active Buzzer	5V, 2300Hz	Audio alerts
LED + 220Ω Resistor	5mm, Red/Green	Visual indication
RFID Cards/Tags	MIFARE Classic 1K	User authentication

8.2 Software Components:

Software / Library	Version	Purpose
Arduino IDE	2.x	Development environment
ESP32 Board Package	v2.0.17	ESP32 hardware support
ESPAsyncWebServer	Latest	Async web server hosting
AsyncTCP	Latest	Async TCP for ESP32
MFRC522 Library	Latest	RC522 RFID control
Adafruit SSD1306	Latest	OLED display driver
Adafruit GFX	Latest	Graphics rendering
LittleFS	Built-in	Flash filesystem for web files
SD Library	Built-in	SD card read/write
SPI Library	Built-in	SPI bus communication

8.3 Development Environment:

- Operating System: Windows 10/11
- IDE: Arduino IDE 2.x with ESP32 Board Package v2.0.17
- LittleFS Uploader Plugin: For uploading web files to ESP32 flash
- Serial Monitor: For debugging and runtime log observation

Browser: Chrome/Firefox for web dashboard testing

9 Development Workflow

9.1 System Setup:

Hardware assembly begins with connecting the RC522 RFID module to VSPI pins, the SD card module to HSPI pins, and the OLED display to I2C pins as per the pin configuration table. LED and buzzer are connected to GPIO 26 and GPIO 4 respectively. Power is supplied via USB to the ESP32. All connections are verified before firmware upload.

The Arduino IDE is configured with the ESP32 board package v2.0.17 (specifically this version to ensure ESPAsyncWebServer compatibility). Required libraries are installed via the Library Manager. The LittleFS uploader plugin is installed for flashing web dashboard files.

9.2 Configuration Process:

Before compilation, the following configuration parameters are set in the firmware: WiFi SSID and password, web server port (default 80), RFID module pin assignments, SD module pin assignments, log file path on SD card, and initial user database entries. The user database is stored in SD card storage as a text-based user database file, which is uploaded separately from the firmware.

9.3 Execution Flow:

Upon startup, the system follows this execution flow:

1. ESP32 boots and initializes all peripherals
2. LittleFS is mounted; web server starts serving dashboard files
3. OLED displays 'System Ready' with IP address
4. Main loop begins: RFID module polled continuously
5. Card detected: UID extracted and database checked
6. OLED updates with result; LED/buzzer activate
7. Event logged to SD card with timestamp
8. Dashboard API updated; client browsers reflect new event

10 Issues Faced & Solutions

The development process encountered several hardware and software challenges. The following table summarizes all issues encountered, their root causes, and the solutions applied:

Issue	Cause	Solution
ESP32 upload failed (wrong boot mode)	ESP32 not in flashing/download mode	Held BOOT button during upload
MFRC522 communication failure	Incorrect pin config and SPI conflict	Corrected RFID pins, separated SPI buses
ESPAsyncWebServer compilation errors	ESP32 board package version incompatibility	Downgraded board package to v2.0.17
LittleFS mounting failed	Corrupted filesystem	Erased flash and re-uploaded LittleFS data
404 webpage error	LittleFS web files not uploaded	Installed LittleFS uploader, uploaded data folder

Issue	Cause	Solution
RFID and SD card not working together	SPI bus conflict between RC522 and SD module	Separated onto VSPI and HSPI buses
SD card mount failed randomly	GPIO15 & GPIO12 boot pin conflicts	Moved SD module to safer HSPI pins
ESP32 upload stopped during flashing	SD module connected to ESP32 boot pins	Changed SD module wiring to safer pins
OLED blank screen	OLED initialization missing in code	Added Adafruit SSD1306 initialization code
LED pin conflict with OLED	GPIO22 used by OLED SCL	Moved LED pin to GPIO26

11 Testing & Results

11.1 Test Scenarios:

The following test scenarios were executed to validate system functionality:

1. Registered Card Scan: Verified access granted response, correct user name/role on OLED, green LED/buzzer pattern, and log entry creation
2. Unregistered Card Scan: Verified access denied response, OLED denied message, red LED/buzzer pattern, and unknown UID log entry
3. Multiple Rapid Scans: Verified system stability under rapid successive card presentations
4. SD Card Log Persistence: Verified logs persist across power cycles
5. Web Dashboard User Add/Remove: Verified real-time database updates via dashboard
6. WiFi Reconnection: Verified system behavior on temporary WiFi disconnection

11.2 Performance Evaluation:

Metric	Target	Achieved
Authentication response time	< 500ms	~200-350ms
OLED update time	< 100ms	~50ms
SD log write time	< 200ms	~80-120ms
Web dashboard update	< 2 seconds	~1-1.5 seconds
WiFi connection time	< 10 seconds	~3-5 seconds
System uptime (continuous)	> 24 hours	Stable, no crashes observed

11.3 Project Status Summary:

Module	Status
RFID Authentication System	Working
SD Card Logging	Working
OLED Display Interface	Working
Web Dashboard	Working
WiFi Connectivity	Working
LED Indicator	Working
Buzzer Alert System	Working
LittleFS File System	Working

12 Security Analysis

12.1 Authentication Security:

The system implements UID-based RFID authentication as the primary security mechanism. While effective for controlled environments, UID-based authentication is vulnerable to card cloning attacks where an attacker copies the UID of a registered card. For enhanced security, future implementations should leverage the MIFARE Classic card's encrypted memory sectors for challenge-response authentication.

12.2 Web Dashboard Security:

The web dashboard is accessible to all devices on the local network without authentication in the current implementation. This is acceptable for controlled lab environments but represents a vulnerability in production deployments. Recommended enhancements include HTTP Basic Authentication or token-based access control for the dashboard API endpoints.

12.3 Physical Security:

The system's physical security depends on the secure placement of the ESP32 hardware. If an attacker gains physical access to the device, they could potentially read the SD card logs or modify the firmware. Enclosure of the hardware in a tamper-evident case and SD card encryption would mitigate these risks in production deployments.

12.4 Audit Trail Integrity:

The SD card logging provides a persistent audit trail of all authentication events. However, the current implementation does not protect log integrity against modification. Production systems should implement cryptographic log signing or write-once storage to ensure tamper-evident audit records.

13 Limitations & Future Enhancements

13.1 Current Limitations:

- Authentication is based on UID only, which is vulnerable to card cloning
- Web dashboard lacks authentication, accessible to all local network users
- No real-time clock (RTC) hardware; timestamps depend on NTP which requires internet
- Single ESP32 cannot handle very high concurrent web requests
- Limited flash storage constrains the size of the user database
- No encrypted communication between dashboard and server (HTTP only)

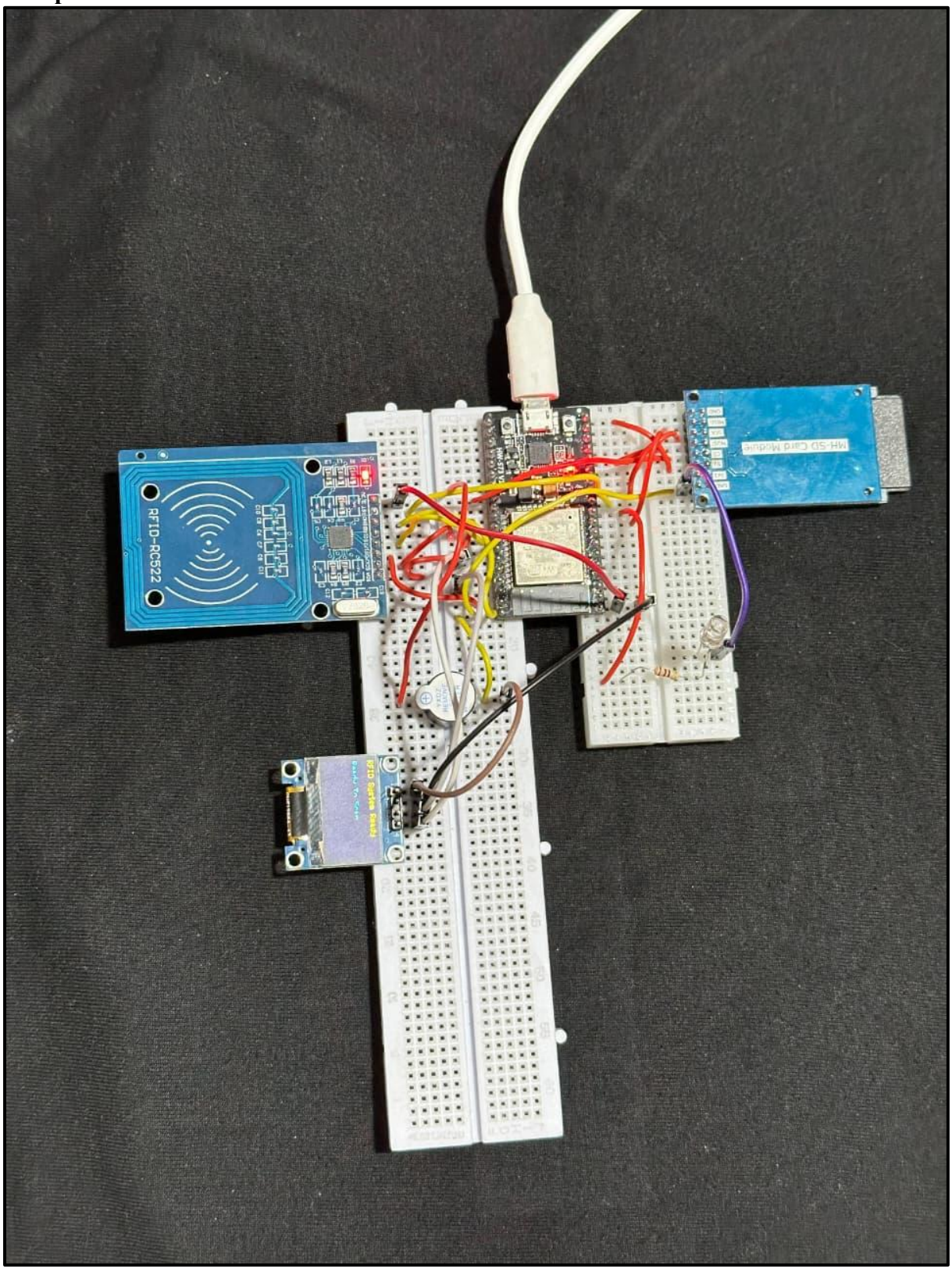
13.2 Future Enhancements:

The following enhancements are planned for future development iterations:

Enhancement	Description
Smart Door Lock Integration	Relay-controlled electronic door lock triggered by access granted events
Firebase Cloud Synchronization	Real-time sync of logs and user database to Firebase for remote access
Telegram Notifications	Instant push notifications for access events via Telegram Bot API
Mobile Application	Dedicated Android/iOS app for user management and monitoring
Biometric Authentication	Fingerprint sensor integration for two-factor authentication
Face Recognition	Camera module integration for face-based access control
Blockchain-based Secure Logs	Immutable audit trail using blockchain technology
Attendance Analytics Dashboard	Advanced charts and analytics for access patterns and attendance tracking

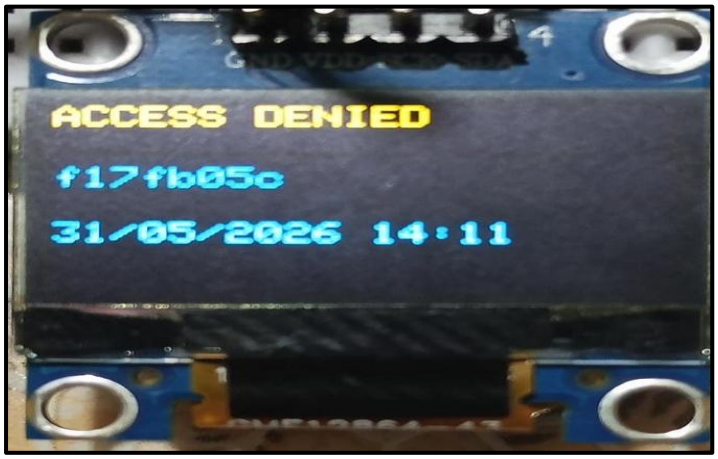
14 Project Implementation Screenshots:

Complete circuit

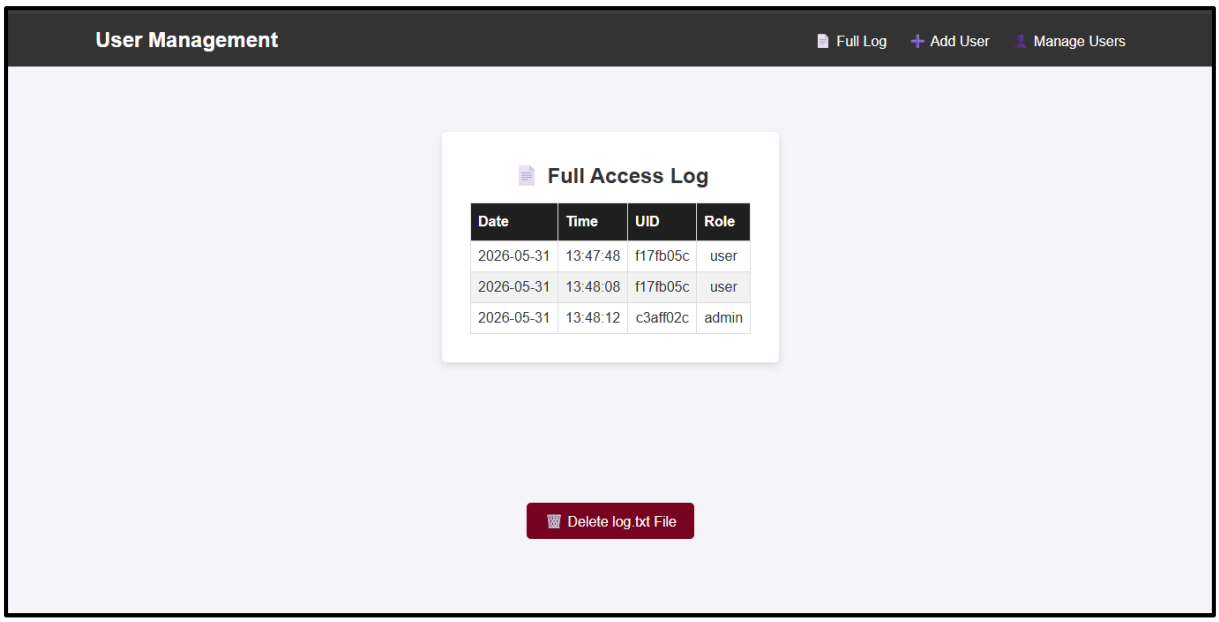


OLED Access Granted

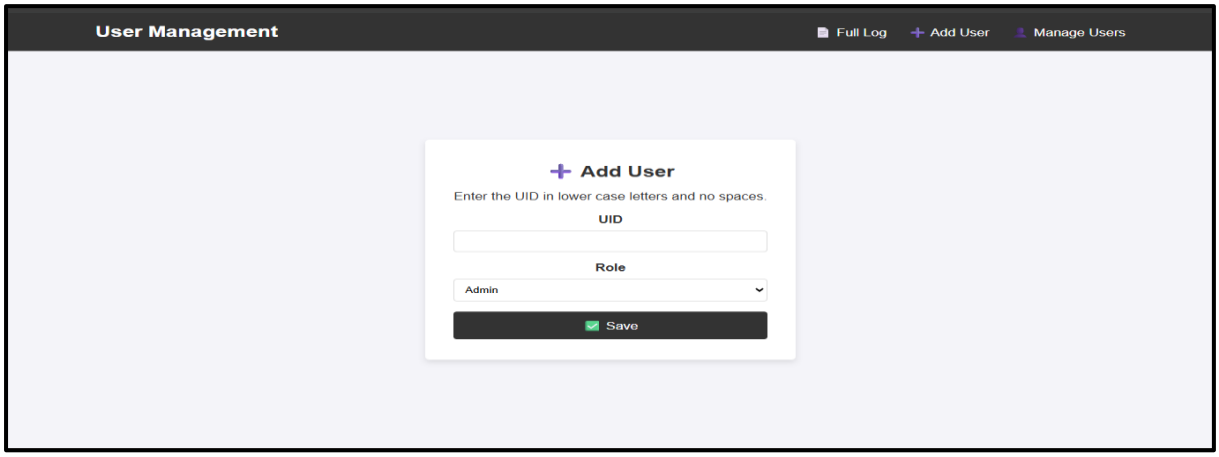
OLED Access Denied



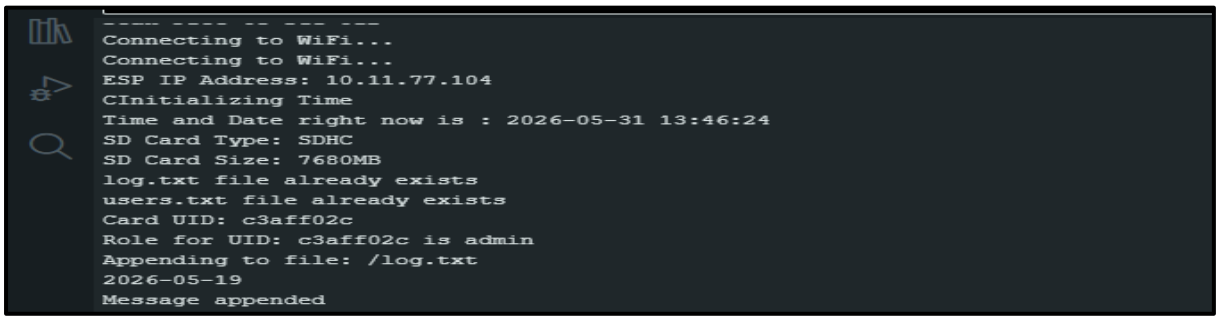
Web dashboard



Add user page



Serial monitor



15 Conclusion:

This project successfully demonstrated the development of a comprehensive IoT-Based Smart RFID User Management and Access Control System using the ESP32 microcontroller. By integrating multiple hardware peripherals and software components, the system achieves a complete access control solution combining RFID authentication, real-time OLED feedback, persistent SD card logging, and a WiFi-hosted web dashboard.

The project provided valuable hands-on experience with embedded systems programming, SPI and I2C communication protocols, IoT web server development, and practical problem-solving in hardware-software integration. Key technical challenges such as dual SPI bus management, LittleFS filesystem configuration, and ESPAsyncWebServer compatibility were successfully resolved.

All system modules including RFID authentication, SD card logging, OLED display, web dashboard, WiFi connectivity, LED indicators, and buzzer alerts are fully functional. The system meets its primary objectives of providing secure, real-time, and persistent access control with remote monitoring capabilities.

The project serves as a strong foundation for future enhancements including cloud integration, mobile application development, biometric authentication, and advanced analytics. It effectively bridges theoretical cybersecurity and IoT concepts with practical implementation, making it a valuable academic and demonstrative contribution.

15. References

[1] Espressif Systems, “ESP32-WROOM-32 Datasheet,” Espressif Systems, 2025. [Online].

Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf

[2] Espressif Systems, “ESP32 Technical Reference Manual,” Espressif Systems, 2025. [Online].

Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[3] NXP Semiconductors, “MFRC522 Standard Performance MIFARE and NTAG Frontend Datasheet,” NXP Semiconductors. [Online].

Available: https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf

[4] Arduino, “Arduino IDE Documentation,” Arduino Documentation. [Online].

Available: https://docs.arduino.cc/software/ide/

[5] Arduino ESP32 Core Documentation, “Arduino Core for ESP32,” Espressif Systems. [Online].

Available: https://docs.espressif.com/projects/arduino-esp32/en/latest/

[6] M. Balboa, “MFRC522 RFID Library for Arduino,” GitHub Repository. [Online].

Available: https://github.com/miguelbalboa/rfid

[7] Adafruit Industries, “Adafruit SSD1306 OLED Display Library,” GitHub Repository. [Online].

Available: https://github.com/adafruit/Adafruit_SSD1306

[8] Adafruit Industries, “Adafruit GFX Graphics Library,” GitHub Repository. [Online].

Available: https://github.com/adafruit/Adafruit-GFX-Library

[9] ESP32Async, “ESPAsyncWebServer Library,” GitHub Repository. [Online].

Available: https://github.com/ESP32Async/ESPAsyncWebServer

[10] ESP32Async, “AsyncTCP Library for ESP32,” GitHub Repository. [Online].

Available: https://github.com/ESP32Async/AsyncTCP

[11] SD Association, “SD Memory Card Specifications,” SD Association. [Online].

Available: https://www.sdcard.org/downloads/pls/

[12] Philips Semiconductors, “The I2C-Bus Specification and User Manual,” NXP Semiconductors. [Online].

Available: https://www.nxp.com/docs/en/user-guide/UM10204.pdf